Report: January

Rishi Vora

2026-01-24

1. Short Summary of Previous Work

In July, I worked on learning about Classical DDPM models and implemented a basic version of it with `PyTorch`. Based on that, we started exploring Quantum DDPM models.

2. Base

We focused on the implementation by Zhang et al. [1]. Here, we take an ensemble of states of our interest, gradually add noise to them via a Quantum Scrambling Circuit (QSC), then train multiple Parameterized Quantum Circuits (PQC) to denoise the ensemble step-by-step.

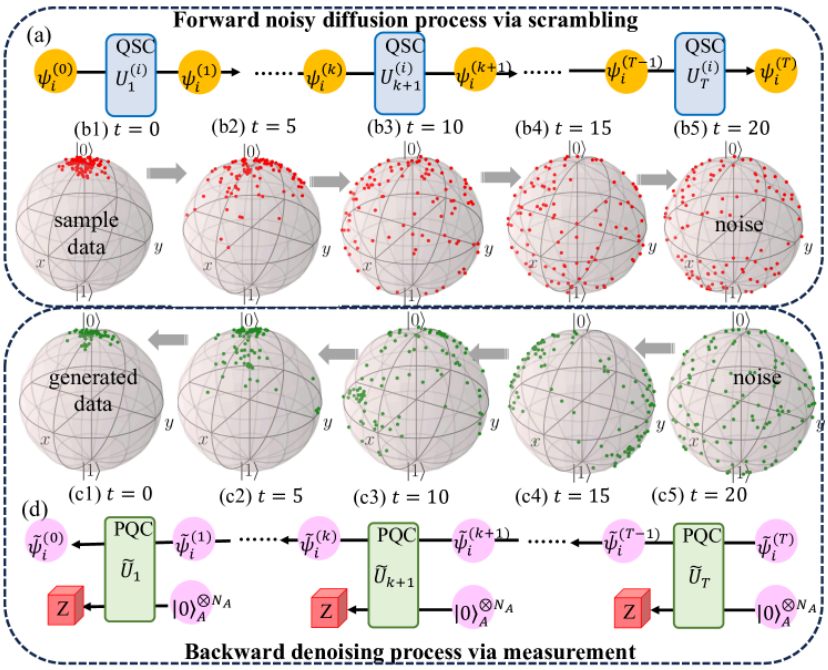

Figure 1: Overview of Quantum DDPM

Credits: [1]

Let there be ensembles of states $\mathcal{E}_1$ and $\mathcal{E}_2$ of size n_1 and n_2 respectively. Two loss functions are used here:

- **Maximum Mean Discrepancy (MMD)**

$$\mathcal{D}_{\text{MMD}}(\mathcal{E}_1, \mathcal{E}_2) = \bar{F}(\mathcal{E}_1, \mathcal{E}_1) + \bar{F}(\mathcal{E}_2, \mathcal{E}_2) - 2\bar{F}(\mathcal{E}_1, \mathcal{E}_2)$$

  where

$$\bar{F}(\mathcal{E}_1, \mathcal{E}_2) = \mathbb{E}_{|\varphi\rangle \sim \mathcal{E}_1, |\psi\rangle \sim \mathcal{E}_2} \ |\langle \varphi | \psi \rangle|^2$$

  - Worked well for simple distribution of states (e.g. states clustered around the ground state)
  - Computationally efficient

- **Wasserstein Distance**

$$\begin{aligned} W_2(\mathcal{E}_1, \mathcal{E}_2) = \min_P \langle P, C \rangle, \\ s.t. \quad P\mathbf{1}_{n_1} = 1, \\ P\mathbf{1}_{n_2} = 1, \\ P_{ij} \geq 0 \end{aligned}$$

  where

$$C_{ij} = 1 - |\langle \varphi_i | \psi_j \rangle|^2$$

  and P is a transport plan.
  - Worked better for complex distributions
  - Computationally expensive

3. My Work

I implemented the Quantum DDPM model as per the above paper using `PyTorch`. I used their own `PyTorch` implementation as a reference and reimplemented it in a much cleaner manner. Being `Python`, it was quite slow to simulate, despite the extensive use of Numpy arrays and `PyTorch` tensors. So I ported the entire implementation to `Julia`. The training time was cut down to a fraction of the original time.

3.1. Experimentation

Architectures

For the distribution of simple states (clustered around the ground state), the original diffusion type model worked well. But Consistency type model [2] worked even better, converging faster and yielding better results.

For distribution of arbitrary states, the Consistency model did not work well.

Loss functions

Apart from the two loss functions mentioned, I experimented with a few other variants:

- **Sinkhorn Distance**
  - Approximates Wasserstein distance with entropic regularization
  - More computationally efficient than Wasserstein distance
- **Inexact Proximal point method for exact Optimal Transport problem (IPOT)**
  - Another approximation for Wasserstein distance but has theoretical guarantees of convergence to the Wasserstein distance [3]
  - Computationally efficient

Optimizers

- Adam optimizer worked well for most cases.
- Rotosolve optimizer showed promise for smaller systems. One key advantage is that it does not require gradient calculations, making it potentially more efficient. More testing is needed.
- [3] hints at a simpler gradient calculation method for the Wasserstein distance, which I am currently testing.

4. Planned Work

Next, I have been tasked with generating Quantum Kicked Rotor states, which requires implementing the model for atleast 9-10 qubits. Given the exponential scaling of quantum state simulation on classical hardware, this requires signification compute power. So I am optimising my code further.

I am also looking into implementing potentially better loss functions (e.g. Sinkhorn divergence) and optimizers (e.g. Quantum Natural Gradient Descent).

Bibliography

[1] B. Zhang, P. Xu, X. Chen, and Q. Zhuang, "Generative quantum machine learning via denoising diffusion probabilistic models," *Phys. Rev. Lett.*, vol. 132, no. 10, p. 100602, Mar. 2024.

[2] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever, "Consistency Models," 2023.

[3] Y. Xie, X. Wang, R. Wang, and H. Zha, "A Fast Proximal Point Method for Computing Exact Wasserstein Distance," 2018.